```
import cron from 'node-cron';
import { CONFIG } from './config.js';
import { initDB, jobExists, insertJob, updateJobStatus, getStats } from './db.js'
import { scrapeNewJobs } from './scraper.js';
import { evaluateJob, passesFilter } from './evaluator.js';
import { generateCoverLetter } from './writer.js';
import { initTelegram, notifyJob, notifyText } from './telegram.js';

initDB();
initTelegram();

console.log('🚀 Karina Job Agent iniciado');
notifyText('🚀 *Agent online* — buscando vagas...');

async function runCycle() {
  const hour = new Date().toLocaleString('en-US', {
    timeZone: CONFIG.runtime.timezone,
    hour: 'numeric',
    hour12: false,
  });

  if (parseInt(hour) < CONFIG.runtime.activeHoursStart ||
      parseInt(hour) >= CONFIG.runtime.activeHoursEnd) {
    console.log(`[${new Date().toISOString()}] Fora do horário ativo, pulando cic
    return;
  }

  console.log(`\n=== Ciclo iniciado ${new Date().toLocaleString()} ===`);

  try {
    const jobs = await scrapeNewJobs();
    console.log(`📋 ${jobs.length} vagas encontradas`);

    let newCount = 0;
    let qualifiedCount = 0;

    for (const job of jobs) {
      if (jobExists(job.id)) continue;
      newCount++;

      const evaluation = await evaluateJob(job);
      console.log(`  [${evaluation.score}/10] ${job.title.slice(0, 60)}`);

      let coverLetter = null;
```

```
    let status = 'rejected';

    if (passesFilter(job, evaluation)) {
      coverLetter = await generateCoverLetter(job, evaluation);
      status = 'pending';
      qualifiedCount++;
    }

    insertJob({
      id: job.id,
      url: job.url,
      title: job.title,
      description: job.description.slice(0, 2000),
      budget_type: job.budget_type,
      budget_min: job.budget_min,
      budget_max: job.budget_max,
      posted_at: new Date().toISOString(),
      match_score: evaluation.score,
      match_reasoning: evaluation.reasoning,
      category: evaluation.category,
      cover_letter: coverLetter,
      status,
    });

    if (status === 'pending' && coverLetter) {
      const fullJob = {
        ...job,
        match_score: evaluation.score,
        match_reasoning: evaluation.reasoning,
        category: evaluation.category,
        cover_letter: coverLetter,
      };
      const msgId = await notifyJob(fullJob);
      updateJobStatus(job.id, 'pending', { telegramMsgId: msgId });
    }

    await new Promise(r => setTimeout(r, 500));
  }

  console.log(`✅ Ciclo terminou: ${newCount} novas, ${qualifiedCount} qualific

  if (qualifiedCount === 0 && newCount > 0) {
    await notifyText(`🔍 ${newCount} vagas novas vistas, nenhuma qualificada ne
  }
} catch (err) {
  console.error('❌ Erro no ciclo:', err);
  await notifyText(`❌ *Erro:* ${err.message}`);
```

```
  }
}

// Roda imediatamente
runCycle();

// Agenda execução periódica
cron.schedule(`*/${CONFIG.runtime.intervalMinutes} * * * *`, runCycle);

// Relatório diário às 20h
cron.schedule('0 20 * * *', async () => {
  const stats = getStats();
  await notifyText(`📊 *Relatório diário*

👁️ Vistas: ${stats.total_seen}
✅ Aplicadas: ${stats.applied}
⏳ Pendentes: ${stats.pending}
⏭️ Puladas: ${stats.skipped}
❌ Rejeitadas automaticamente: ${stats.rejected}
📊 Match médio: ${stats.avg_match?.toFixed(1) || 'N/A'}`);
}, { timezone: CONFIG.runtime.timezone });

process.on('SIGINT', async () => {
  await notifyText('🛑 Agent desligado manualmente');
  process.exit(0);
});
```